

# COL783 Assignment-3

Ujjawal Sharma (2022CS51137), Sarthak Panda (2022CS51217)

## Question 1: Image Blending using Gaussian and Laplacian Pyramids

We implement multi-resolution image blending using Gaussian and Laplacian pyramids to seamlessly stitch an apple and an orange image. The process involves constructing pyramids for both images and a binary mask, blending at each level in the Laplacian domain, and reconstructing the final image with smooth transitions.

We use two input images: **q1\_apple.jpg** ( $f$ ) and **q1\_orange.jpg** ( $g$ ), both resized to the same dimensions. A 1D Gaussian kernel  $w = \frac{1}{16}[1, 4, 6, 4, 1]$  is used for blurring and interpolation.

### Part A: Gaussian Pyramid Construction

The Gaussian pyramid is built by repeatedly applying:

1. 2D separable convolution with kernel  $w$  in both directions.
2. Downsampling by a factor of 2 (taking every alternate pixel).

Let  $G_0 = f$  (or  $g$ ). Then:

$$G_k = \downarrow_2 (w * w * G_{k-1}), \quad k = 1, 2, \dots, L - 1$$

We construct  $L = 6$  levels.

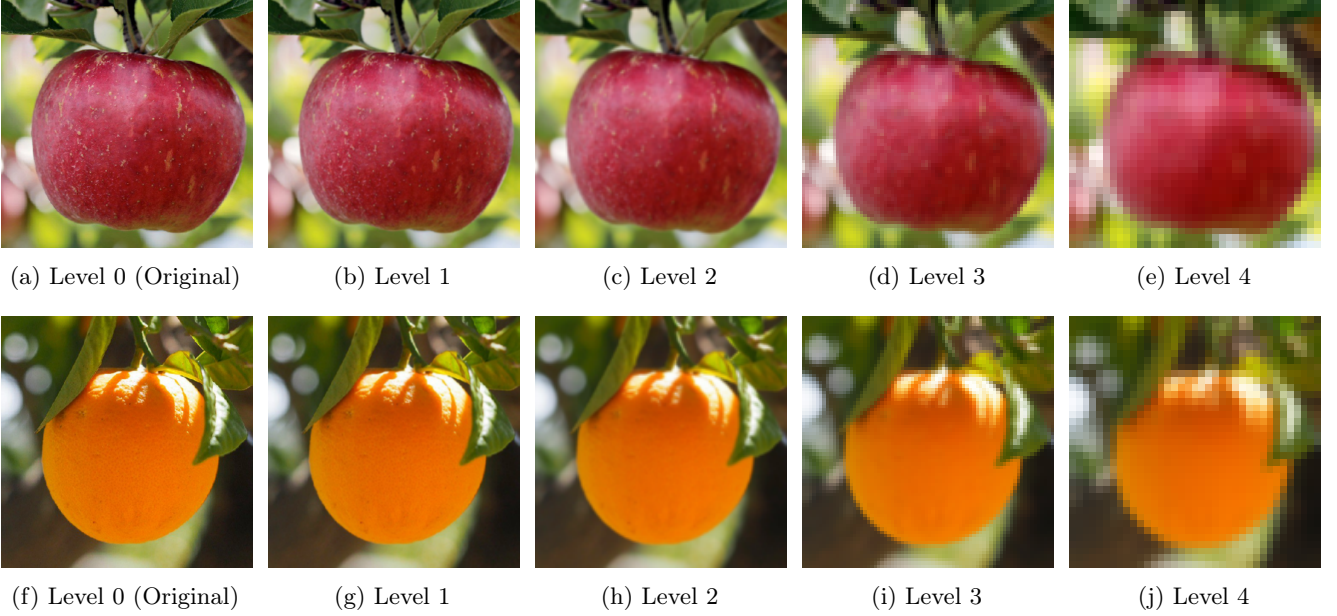


Figure 1: **Gaussian Pyramids (Part A)**. Top row: Apple ( $f$ ) from original to level 4. Bottom row: Orange ( $g$ ) from original to level 4. Each level is half the resolution of the previous.

### Part B: Laplacian Pyramid and Perfect Reconstruction

The Laplacian pyramid  $L_k$  captures band-pass details:

$$L_k = G_k - \text{PREDICT}(G_{k+1}), \quad k = 0, 1, \dots, L - 2$$

$$L_{L-1} = G_{L-1}$$

Prediction is done via upsampling: insert zeros, scale by 4, and convolve with  $w \times w$ :

$$\text{PREDICT}(G_{k+1}) = 4 \cdot (w * w * \uparrow_2 G_{k+1})$$

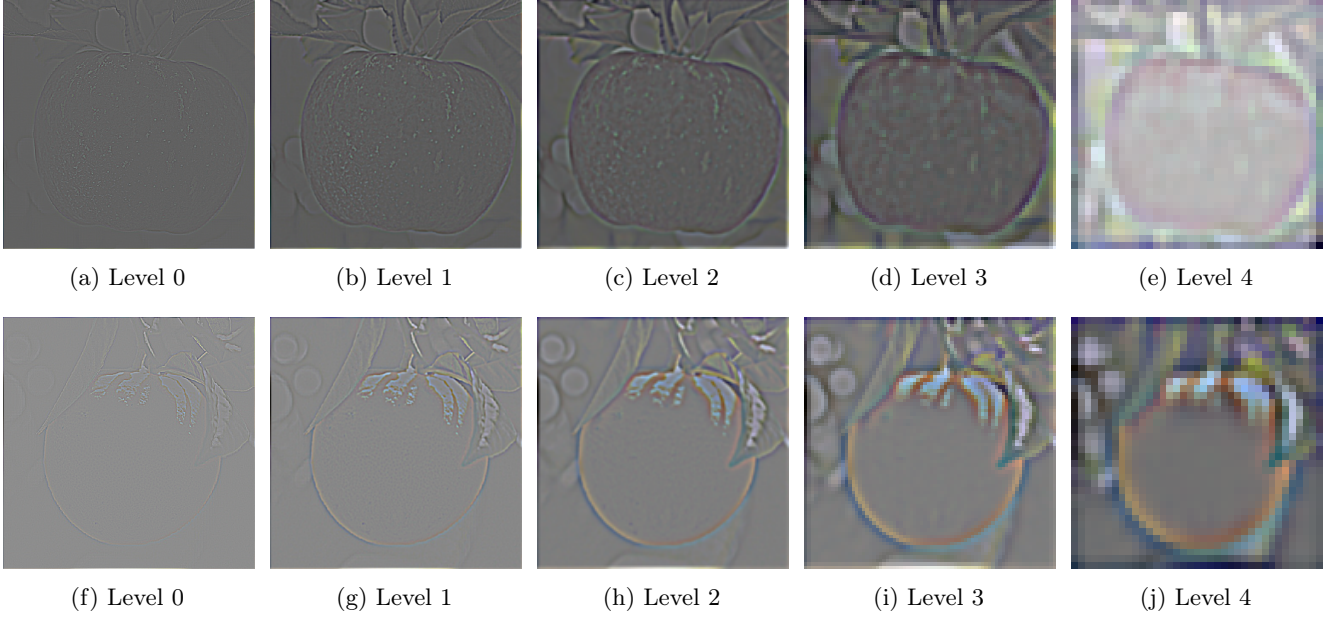


Figure 2: **Laplacian Pyramids (Part B)**. Top: Apple detail bands. Bottom: Orange detail bands. The topmost level (not shown) is the low-resolution residual.

Reconstruction Verification: Starting from  $L_{L-1}$ , we recursively upsample and add:

$$\hat{G}_k = L_k + \text{PREDICT}(\hat{G}_{k+1})$$

Final reconstructed image  $\hat{f}$  matches  $f$  exactly (within floating-point error).

| Metric           | Value    |
|------------------|----------|
| Max Pixel Error  | 1        |
| Mean Pixel Error | 0.000475 |

Table 1: Reconstruction error for apple image (Part B). Near-zero error confirms perfect reconstruction.

### Part C: Hard Transition via Binary Mask

A binary mask  $m$  is manually defined with a vertical split:

$$m(x, y) = \begin{cases} 0 & x < w/2 \\ 1 & x \geq w/2 \end{cases}$$

Hard blend:

$$h = (1 - m) \cdot f + m \cdot g$$



(a) Binary Mask  $m$



(b) Hard Transition Blend

Figure 3: **Hard Transition (Part C)**. Left: Binary mask (black = apple, white = orange). Right: Sharp seam visible along the center.

## Part D: Smooth Blending using Pyramid Blending

We construct a Gaussian pyramid of the mask  $m$ , denoted  $M_k$ . At each level  $k$ , blend Laplacian coefficients:

$$B_k = (1 - M_k) \cdot L_k^f + M_k \cdot L_k^g$$

Reconstruct the final image from  $\{B_k\}$ .

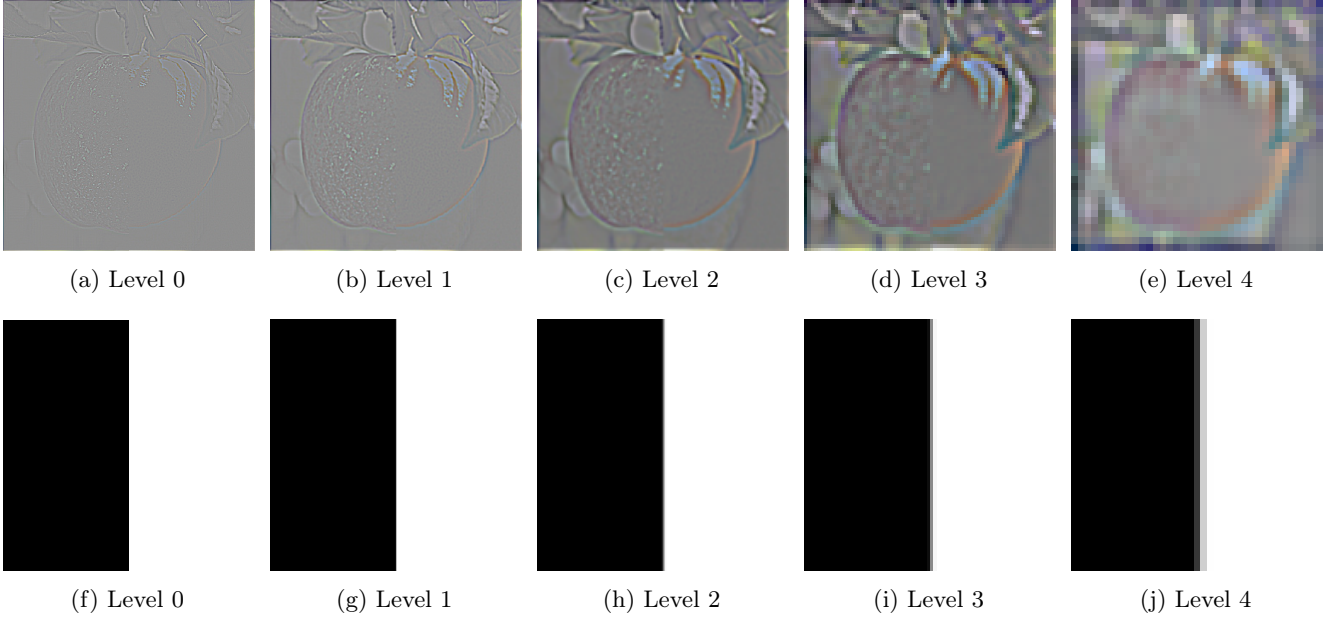


Figure 4: **Pyramid Blending Components (Part D)**. Top: Blended Laplacian levels. Bottom: Gaussian pyramid of the mask (smoothed transition weights).



Figure 5: **Final Smooth Blend (Part D)**. Seamless transition between apple (left) and orange (right). No visible seam due to multi-scale blending.

The resulting hybrid image combines the left half of the apple and the right half of the orange with no visible boundary, validating the power of pyramid-based blending.

Key functions:

- `construct_gaussian_pyramid`: Uses `cv2.sepFilter2D` + downsampling.
- `upsample_and_convolve`: Implements zero-insertion  $\uparrow_2$  followed by  $4 \times (w * w * \cdot)$ .
- `blend_pyramids`: Level-wise weighted sum using Gaussian mask pyramid.

All operations are performed on pixel arrays as required.

## Question 2: Haar Wavelet Transform – Orthogonality, Implementation, and Sparsity

### Part A: Mathematical Proof of Haar Transform Orthogonality

We prove that the 1D Haar wavelet transform is norm-preserving (i.e.,  $\|t\| = \|f\|$ ) for any signal  $f \in \mathbb{R}^N$  with  $N = 2^k$ . This establishes that the transform is orthogonal and hence invertible.

Recursive definition: let  $N = 2^k$  ( $k \geq 0$ ).

Base:  $k = 0$  ( $N = 1$ ):  $\text{haar}(f) = f$  and  $\|\text{haar}(f)\| = \|f\|$ .

For  $k \geq 1$  ( $N \geq 2$ ) define low/high arrays  $c, d \in \mathbb{R}^{N/2}$  by

$$c_i = \frac{f_{2i} + f_{2i+1}}{\sqrt{2}}, \quad d_i = \frac{f_{2i} - f_{2i+1}}{\sqrt{2}}, \quad i = 0, \dots, \frac{N}{2} - 1,$$

and the recursive Haar transform

$$\text{haar}(f) = [\text{haar}(c), d].$$

One-step energy relation (algebraic identity):

$$\begin{aligned} \|c\|^2 + \|d\|^2 &= \sum_{i=0}^{\frac{N}{2}-1} \left( \frac{(f_{2i} + f_{2i+1})^2}{2} + \frac{(f_{2i} - f_{2i+1})^2}{2} \right) \\ &= \sum_{i=0}^{\frac{N}{2}-1} \frac{(f_{2i} + f_{2i+1})^2 + (f_{2i} - f_{2i+1})^2}{2} = \sum_{i=0}^{\frac{N}{2}-1} (f_{2i}^2 + f_{2i+1}^2) \\ &= \sum_{n=0}^{N-1} f_n^2 = \|f\|^2. \end{aligned}$$

Inductive step: assume for length  $N/2$  that  $\|\text{haar}(c)\| = \|c\|$ .

Then  $\|\text{haar}(f)\|^2 = \|\text{haar}(c)\|^2 + \|d\|^2 = \|c\|^2 + \|d\|^2 = \|f\|^2$ .

Therefore by induction on  $k$ ,  $\|\text{haar}(f)\| = \|f\|$  for all  $N = 2^k$ .

Numerical Verification on Test Signal  $f = [1, 2, 3, 4, 5, 6, 7, 8]$ :

$$\begin{aligned} \|f\| &= 14.282857, \quad \|\text{haar}(f)\| = 14.282857, \quad \|\|f\| - \|t\|\| = 0.00 \\ \text{Reconstruction error} &= \|f - \text{ihaar}(t)\| = 3.95 \times 10^{-15} \end{aligned}$$

The zero difference in norms and near-zero reconstruction error confirm the theoretical result and implementation correctness.

This orthogonality extends naturally to the 2D separable Haar transform used in Part B.

### Part B: 2D Haar Transform and Visualization

We implement the 2D Haar wavelet transform by applying the 1D Haar transform first along all rows, then along all columns of the resulting matrix. The inverse transform follows the reverse order: columns then rows.

Let  $f \in \mathbb{R}^{N \times N}$  be the input image. The 2D forward transform  $t = \mathcal{H}_{2D}(f)$  is:

$$t = \mathcal{H}_{1D}^{\text{col}}(\mathcal{H}_{1D}^{\text{row}}(f))$$

where  $\mathcal{H}_{1D}^{\text{row}}$  applies the 1D Haar transform to each row, and  $\mathcal{H}_{1D}^{\text{col}}$  applies it to each column of the intermediate result.

Since the 1D Haar transform is orthonormal (as proven in Part A), the 2D transform is also orthonormal:

$$\|t\|_F = \|f\|_F$$

where  $\|\cdot\|_F$  is the Frobenius norm.

We apply this to a  $256 \times 256$  grayscale image (middle row used in Part A). The Haar coefficients contain both positive and negative values. To visualize  $t$ , we normalize and map it to RGB as follows: - Negative values  $\rightarrow$  red channel (using power-law mapping for contrast) - Positive values  $\rightarrow$  blue channel - Zero  $\rightarrow$  black

This mapping ensures high-magnitude details (edges, textures) are clearly visible.

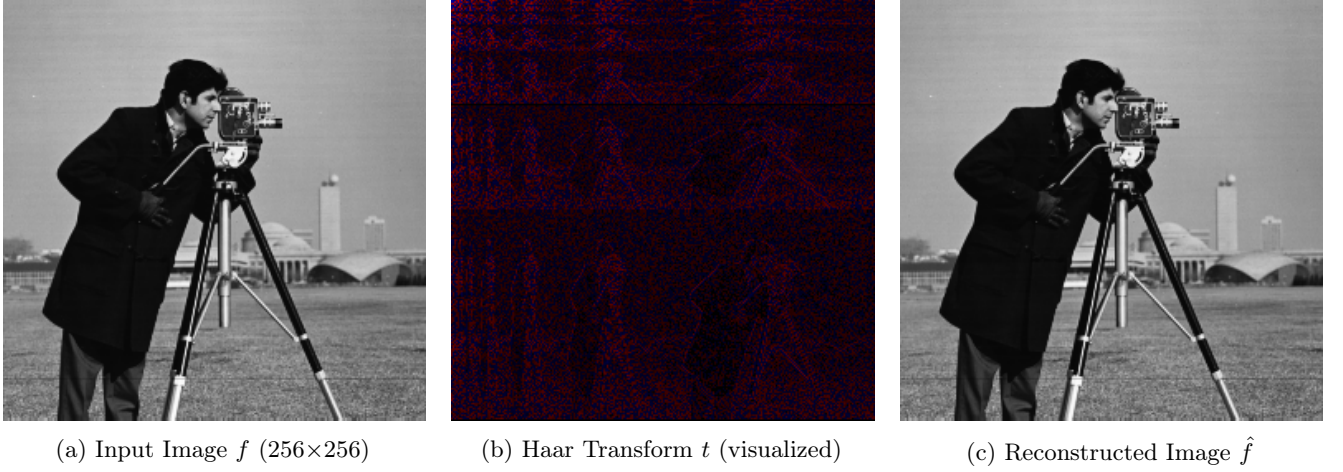


Figure 6: **2D Haar Transform and Reconstruction (Part B)**. Left: Original grayscale image. Center: Haar coefficients visualized (red = negative, blue = positive, black = zero). Right: Inverse transform recovers the original image exactly ( $\text{PSNR} \rightarrow \infty$ ,  $\text{error} < 10^{-12}$ ).

The reconstruction error is negligible (on the order of floating-point precision), confirming perfect invertibility of the 2D Haar transform.

### Part C: Sparsity and Compression via Coefficient Selection

The Haar transform concentrates most image energy into a small number of large-magnitude coefficients. We explore this sparsity by: **(1)** Retaining only the top  $N^2/16$  (1/16) largest absolute coefficients and zeroing the rest. **(2)** Reconstructing the image from this sparse representation. **(3)** Comparing with downsampling the image to  $N/4 \times N/4$  (also using  $N^2/16$  coefficients) and upscaling via Lanczos interpolation. We repeat for top  $N^2/256$  (1/256) coefficients vs. downsampling to  $N/16 \times N/16$ .

#### Case 1: $N^2/16$ Coefficients (1/16 of total)



Figure 7: **Sparsity Comparison (1/16 Coefficients)**. Left: Downsampling + interpolation introduces blockiness and blurs edges. Right: Haar-based approximation preserves sharp edges and textures with far better visual fidelity despite using the same number of coefficients.

**Case 2:  $N^2/256$  Coefficients (1/256 of total)**



(a) Downsampled to  $16 \times 16 \rightarrow$  Upscaled



(b) Haar: Top  $N^2/256$  Coefficients

Figure 8: **Sparsity Comparison (1/256 Coefficients)**. Left: Downsampling loses nearly all detail. Right: Haar reconstruction retains major structural elements (person, tripod, horizon), demonstrating superior energy compaction.

Table 2: **Comparison of Sparsity and Reconstruction Quality (Part C)**

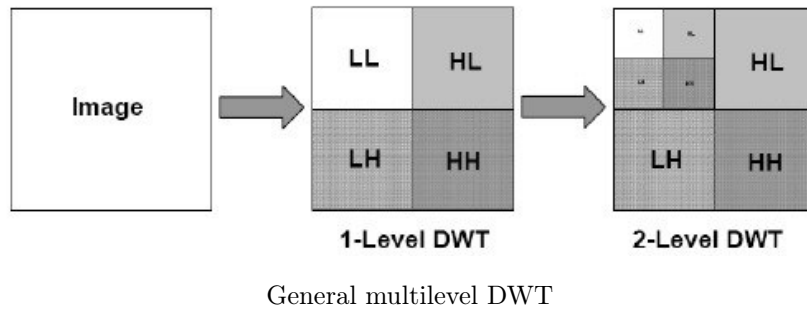
| Method                | # Coefficients | Visual Quality        | Edge Preservation | Energy Compaction |
|-----------------------|----------------|-----------------------|-------------------|-------------------|
| Downsampling          | $N^2/16$       | Poor (blocky, blurry) | No                | Poor              |
| Haar (top $N^2/16$ )  | $N^2/16$       | <b>High</b>           | <b>Yes</b>        | <b>Excellent</b>  |
| Downsampling          | $N^2/256$      | Very poor             | No                | Poor              |
| Haar (top $N^2/256$ ) | $N^2/256$      | Moderate              | Partial           | <b>Excellent</b>  |

**Conclusion:** The Haar transform sparsifies natural images effectively. Even with only 1/256 of the coefficients, it retains recognizable structure i.e. far superior to uniform downsampling. This superior energy compaction forms the foundation of wavelet-based compression (e.g., JPEG 2000).

————— **End of Part-I** —————

### Question 3: Wavelet Denoising [*Optional Solved*]

In general the discrete wavelet transform involves taking recursive transforms only in the upper-left quadrant.



Here we keep the input image  $f$  and  $g$  used for following parts.



Input Image ( $f$ )



20dB Noisy Input Image ( $g$ )

#### Part-A : Haar wavelet transform

##### Haar Forward Transform :

---

##### Algorithm 1 1D Forward Haar Wavelet Transform

---

**Require:** Signal  $x \in \mathbb{R}^n$  (even  $n$ )

**Ensure:** Coefficients  $c \in \mathbb{R}^n$  where  $c = [a|d]$

```

1: Step 1: Pairwise Sum and Difference
2: for  $k = 0$  to  $n/2 - 1$  do
3:    $a_k \leftarrow \frac{x_{2k} + x_{2k+1}}{\sqrt{2}}$  ▷ Approximation
4:    $d_k \leftarrow \frac{x_{2k} - x_{2k+1}}{\sqrt{2}}$  ▷ Detail
5: end for
6: return  $c \leftarrow [a_0, \dots, a_{n/2-1}, d_0, \dots, d_{n/2-1}]$ 

```

---



---

##### Algorithm 2 2D Single-Level Forward Haar Wavelet Transform

---

**Require:** Image  $I \in \mathbb{R}^{m \times n}$

**Ensure:** Coefficient image  $C \in \mathbb{R}^{m \times n}$

```

1: Step 1: Row-wise 1D Transforms
2: for  $i = 0$  to  $m - 1$  do
3:    $I[i, :] \leftarrow \text{Forward1DHaar}(I[i, :])$ 
4: end for
5: Step 2: Column-wise 1D Transforms
6: for  $j = 0$  to  $n - 1$  do
7:    $I[:, j] \leftarrow \text{Forward1DHaar}(I[:, j])$ 
8: end for
9: return  $I$ 

```

---

## Haar Inverse Transform :

---

### Algorithm 3 1D Inverse Haar Wavelet Transform

---

**Require:** Coefficients  $c = [a|d]$ , each of length  $n/2$

**Ensure:** Reconstructed  $x \in \mathbb{R}^n$

```

1: for  $k = 0$  to  $n/2 - 1$  do
2:    $x_{2k} \leftarrow \frac{a_k + d_k}{\sqrt{2}}$ 
3:    $x_{2k+1} \leftarrow \frac{a_k - d_k}{\sqrt{2}}$ 
4: end for
5: return  $x$ 

```

---



---

### Algorithm 4 2D Single-Level Inverse Haar Wavelet Transform

---

**Require:** Coefficients  $C \in \mathbb{R}^{m \times n}$

**Ensure:** Reconstructed image  $I \in \mathbb{R}^{m \times n}$

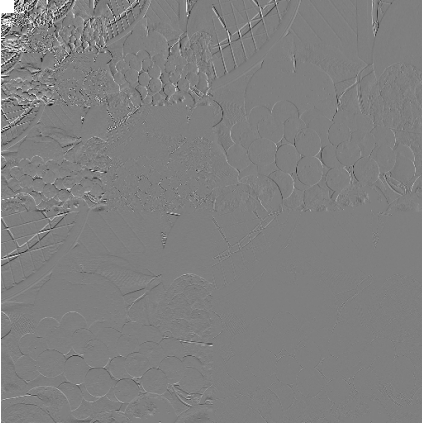
```

1: Step 1: Column-wise Inverse 1D Transforms
2: for  $j = 0$  to  $n - 1$  do
3:    $C[:, j] \leftarrow \text{Inverse1DHaar}(C[:, j])$ 
4: end for
5: Step 2: Row-wise Inverse 1D Transforms
6: for  $i = 0$  to  $m - 1$  do
7:    $C[i, :] \leftarrow \text{Inverse1DHaar}(C[i, :])$ 
8: end for
9: return  $C$ 

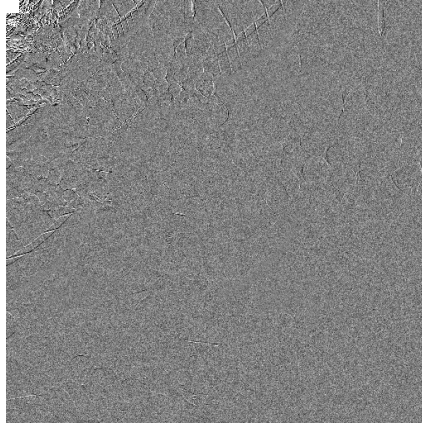
```

---

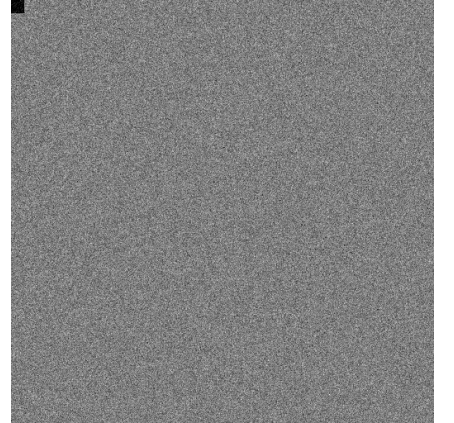
At this point extending to multilevel transform is pretty easy, we just need to re-curse (or recursively build in case of inverse) on LL or top-corner region. Visual Results of Haar Coefficients obtained 5 level wavelet transform is kept below. For analysis purposes we have also kept the histogram of detail coefficient's values.



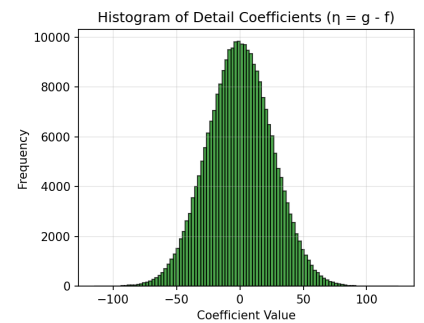
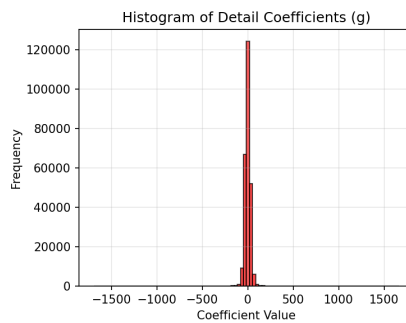
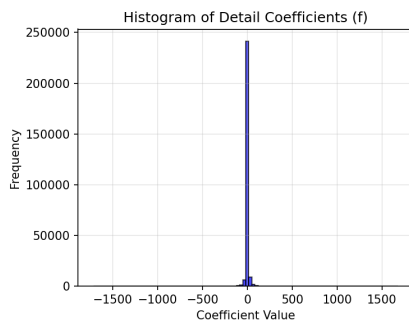
Haar Wavelet Transform of  $f$



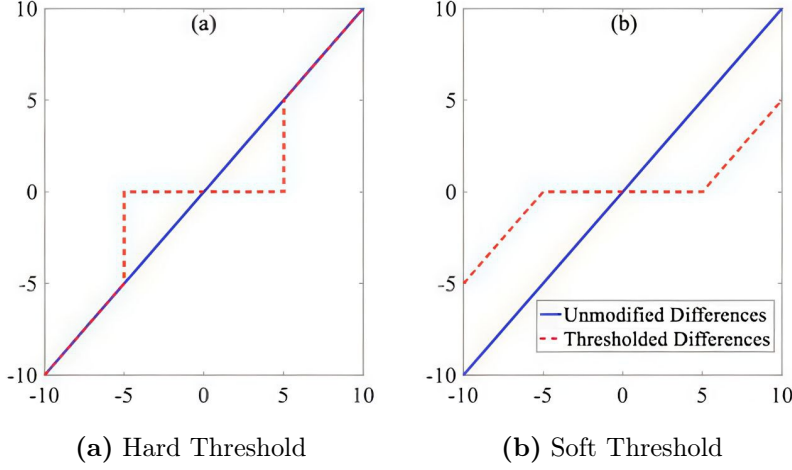
Haar Wavelet Transform of  $g$



Haar Wavelet Transform of  $\eta (= g - f)$



## Part-B : Hard and Soft thresholding



$$H_{\tau}(x) = \begin{cases} x, & |x| \geq \tau, \\ 0, & |x| < \tau. \end{cases}$$

$$H_{\tau}(\mathbf{C}) = (H_{\tau}(c_{ij}))_{i,j}$$

$$S_{\tau}(x) = \text{sign}(x) \max\{|x| - \tau, 0\}$$

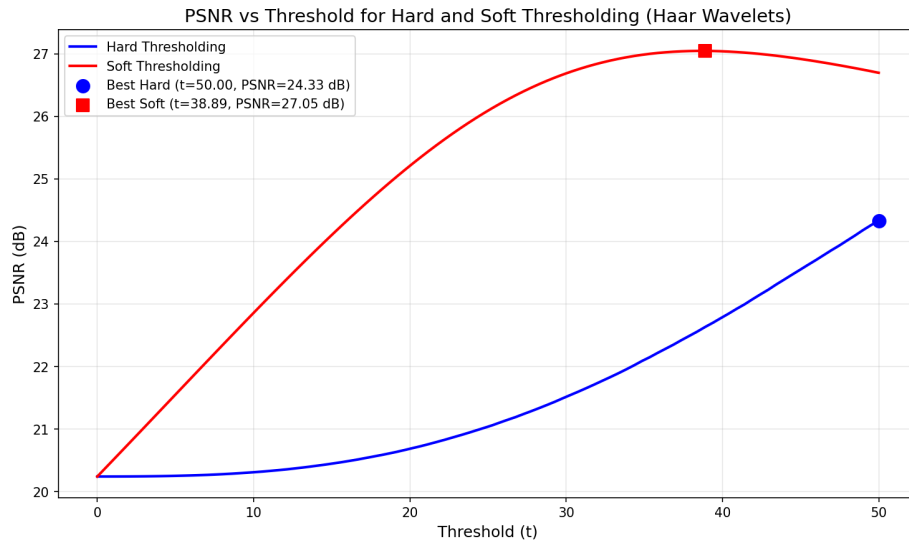
$$S_{\tau}(\mathbf{C}) = (S_{\tau}(c_{ij}))_{i,j}$$



Hard Denoised (Haar)



Soft Denoised (Haar)



If we observe the results, the hard-denoised images tend to preserve coefficient magnitudes and step intensities, so the overall intensity and edge contrast stay closer to the original. However, they retain more high-frequency residual noise and show abrupt, blocky artifacts. In contrast, the soft-denoised images provide stronger noise suppression (often resulting in lower MSE) by shrinking coefficients i.e. noise coefficients near the threshold are reduced instead of being kept as in hard thresholding. This comes at the cost of bias that lowers overall intensity, softens or shrinks sharp edges, and introduces block-like smoothing due to Haar's piecewise-constant nature.

## Part-C : Daubechies(db2) wavelet transform

### DB2 Wavelet Coefficients :

The Daubechies-2 (db2) wavelet is defined by two filter banks:

**Low-pass filter (Scaling function):**

$$h = \left[ \frac{1 + \sqrt{3}}{4\sqrt{2}}, \frac{3 + \sqrt{3}}{4\sqrt{2}}, \frac{3 - \sqrt{3}}{4\sqrt{2}}, \frac{1 - \sqrt{3}}{4\sqrt{2}} \right]$$

**High-pass filter (Wavelet function):**

$$g = [h_3, -h_2, h_1, -h_0]$$

These filters satisfy the orthonormality conditions:

$$\sum_k h_k^2 = 1, \quad \sum_k g_k^2 = 1, \quad \sum_k h_k g_k = 0$$

### DB2 Forward Transform :

---

#### Algorithm 5 1D Forward DB2 Wavelet Transform

---

**Require:** Signal  $x \in \mathbb{R}^n$ , filters  $h, g \in \mathbb{R}^4$ , mode = ‘periodic’

**Ensure:** Coefficients  $c \in \mathbb{R}^n$  where  $c = [c_A | c_D]$

```

1: Step 1: Periodic Extension
2:  $x_{ext} \leftarrow [x_{n-3}, x_{n-2}, x_{n-1}, x_0, x_1, \dots, x_{n-1}, x_0, x_1, x_2]$ 
3:
4: Step 2: Convolution with Filters
5: for  $i = 0$  to  $n - 1$  do
6:   Compute approximation coefficient:
7:    $a_i = \sum_{j=0}^3 h_j \cdot x_{ext}[i + 3 - j]$ 
8:
9:   Compute detail coefficient:
10:   $d_i = \sum_{j=0}^3 g_j \cdot x_{ext}[i + 3 - j]$ 
11: end for
12:
13: Step 3: Downsampling (Decimation by 2)
14:  $c_A \leftarrow [a_0, a_2, a_4, \dots, a_{n-2}]$  ▷ Keep even-indexed samples
15:  $c_D \leftarrow [d_0, d_2, d_4, \dots, d_{n-2}]$  ▷ Keep even-indexed samples
16:
17: Step 4: Concatenate
18: return  $c \leftarrow [c_A | c_D]$  ▷ Length  $n/2 + n/2 = n$ 

```

---



---

#### Algorithm 6 2D Single-Level Forward DB2 Transform

---

**Require:** Image  $I \in \mathbb{R}^{m \times n}$ , filters  $h, g \in \mathbb{R}^4$

**Ensure:** Transformed image  $T \in \mathbb{R}^{m \times n}$  with four subbands

```

1: Step 1: Row-wise Transform
2: Initialize  $T_{temp} \in \mathbb{R}^{m \times n}$ 
3: for  $i = 0$  to  $m - 1$  do
4:    $T_{temp}[i, :] \leftarrow \text{ForwardTransform1D}(I[i, :], h, g)$ 
5: end for
6:
7: Step 2: Column-wise Transform
8: Initialize  $T \in \mathbb{R}^{m \times n}$ 
9: for  $j = 0$  to  $n - 1$  do
10:   $T[:, j] \leftarrow \text{ForwardTransform1D}(T_{temp}[:, j], h, g)$ 
11: end for
12: return  $T$ 

```

---

## DB2 Inverse Transform :

---

### Algorithm 7 1D Inverse DB2 Wavelet Transform

---

**Require:** Coefficients  $c \in \mathbb{R}^n$  where  $c = [c_A | c_D]$ , filters  $h, g \in \mathbb{R}^4$

**Ensure:** Reconstructed signal  $x \in \mathbb{R}^n$

```

1: Step 1: Split Coefficients
2:  $c_A \leftarrow c[0 : n/2]$  ▷ Approximation coefficients
3:  $c_D \leftarrow c[n/2 : n]$  ▷ Detail coefficients
4:
5: Step 2: Create Synthesis Filters
6:  $h_{syn} \leftarrow [h_3, h_2, h_1, h_0]$  ▷ Reverse of  $h$ 
7:  $g_{syn} \leftarrow [g_3, g_2, g_1, g_0]$  ▷ Reverse of  $g$ 
8:
9: Step 3: Upsampling (Insert zeros)
10: Initialize  $c_A^{up} \in \mathbb{R}^n$  with zeros
11: Initialize  $c_D^{up} \in \mathbb{R}^n$  with zeros
12: for  $k = 0$  to  $n/2 - 1$  do
13:    $c_A^{up}[2k] \leftarrow c_A[k]$ 
14:    $c_D^{up}[2k] \leftarrow c_D[k]$ 
15: end for
16:
17: Step 4: Convolution and Reconstruction
18: Initialize  $x \in \mathbb{R}^n$  with zeros
19: for  $i = 0$  to  $n - 1$  do
20:   for  $k = 0$  to  $3$  do
21:      $idx \leftarrow (i - k) \bmod n$  ▷ Circular indexing
22:      $x[i] \leftarrow x[i] + h_{syn}[k] \cdot c_A^{up}[idx]$ 
23:      $x[i] \leftarrow x[i] + g_{syn}[k] \cdot c_D^{up}[idx]$ 
24:   end for
25: end for
26:
27: Step 5: Phase Alignment
28:  $x \leftarrow \text{circshift}(x, -(|h| - 1))$  ▷ Roll by -3 positions
29: return  $x$ 

```

---



---

### Algorithm 8 2D Single-Level Inverse DB2 Transform

---

**Require:** Coefficients  $T \in \mathbb{R}^{m \times n}$ , filters  $h, g \in \mathbb{R}^4$

**Ensure:** Reconstructed image  $I \in \mathbb{R}^{m \times n}$

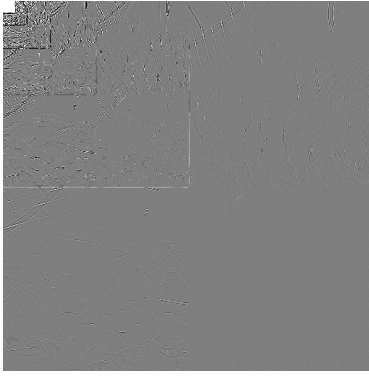
```

1: Step 1: Column-wise Inverse Transform
2: Initialize  $T_{temp} \in \mathbb{R}^{m \times n}$ 
3: for  $j = 0$  to  $n - 1$  do
4:    $T_{temp}[:, j] \leftarrow \text{InverseTransform1D}(T[:, j], h, g)$ 
5: end for
6:
7: Step 2: Row-wise Inverse Transform
8: Initialize  $I \in \mathbb{R}^{m \times n}$ 
9: for  $i = 0$  to  $m - 1$  do
10:   $I[i, :] \leftarrow \text{InverseTransform1D}(T_{temp}[i, :], h, g)$ 
11: end for
12: return  $I$ 

```

---

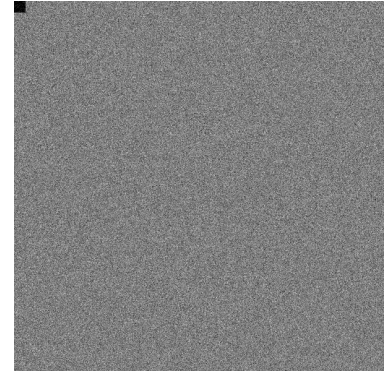
Extending these results to multiple levels is straightforward. For forward encoding, recursively apply the transform to the upper-left (LL) block at each level. For decoding, start at the coarsest level and recursively decode progressively larger LL blocks until you recover the full image. Below, we discuss the results of DB2 wavelet denoising.



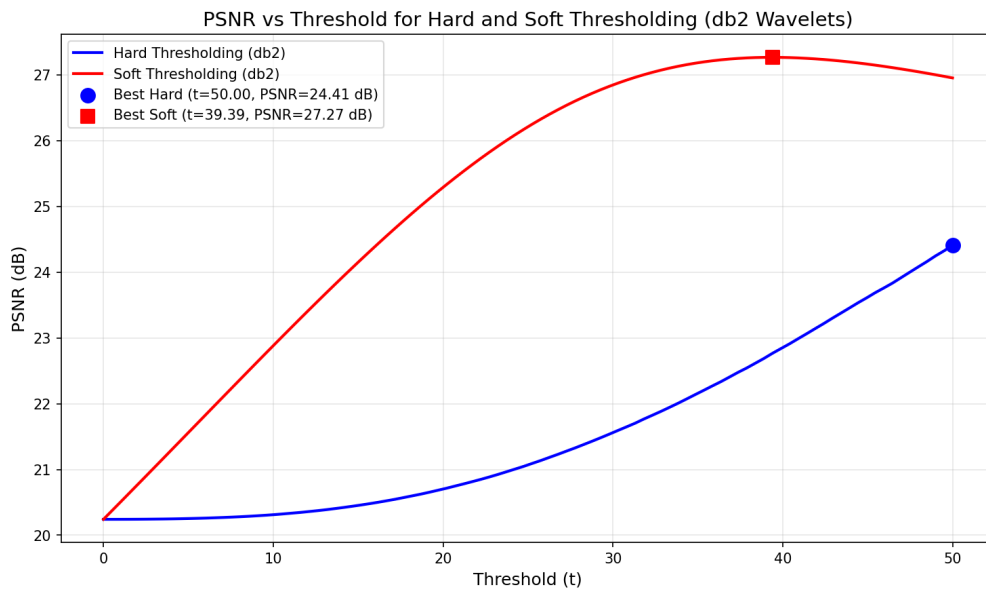
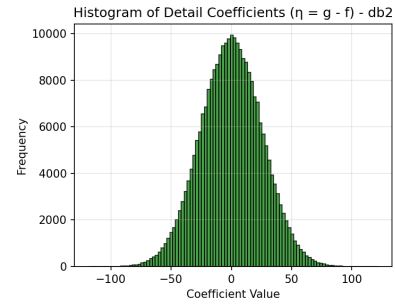
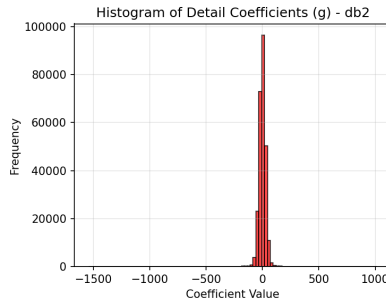
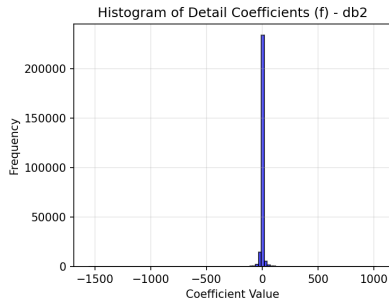
DB2 Wavelet Transform of  $f$



DB2 Wavelet Transform of  $g$



DB2 Wavelet Transform of  $\eta(g-f)$



Hard Denoised (DB2)



Soft Denoised (DB2)

The best hard-threshold denoised image has a PSNR of about 24.41 dB, while soft-thresholding reaches about 27.27 dB. Both results are improvements over the Haar-based denoising for the same 20 dB noisy input. The Haar outputs exhibit more blocky artifacts, whereas DB2 gives smoother results.

## Question 4: Block DCT Coding

In this section we start with images whose height and width are both divisible with 16. **f1** is more of a cartoonish image with large constant intensity image while **f2** is like actual photograph.



Cartoon Image ( $f1$ ; 352x624)



Photographic Image ( $f2$ ; 560x848)

### Part-A : Block DCT & Inverse Block DCT

In this section we implement DCT transform and it's Inverse using SciKit's internal function. Here we keep the results, it's almost same as input.



$f1$ 's reconstruction (PSNR: 317.51 dB)



$f2$ 's reconstruction (PSNR: 318.79 dB)

### Part-B : Lossy Compression Scheme

---

#### Algorithm 9 DCT Coefficient Thresholding to Meet Target PSNR

---

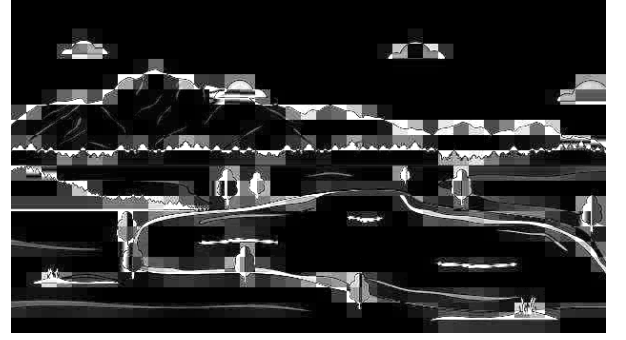
**Require:**  $X \in \mathbb{R}^N$  (DCT coeffs),  $\text{PSNR}_{\text{target}}$ ,  $I_{\text{max}}$

- 1:  $N \leftarrow$  number of coefficients ( $\text{block\_size}^2$ )
  - 2:  $\text{MSE} \leftarrow I_{\text{max}}^2 / 10^{\text{PSNR}_{\text{target}}/10}$
  - 3:  $E_{\text{target}} \leftarrow \text{MSE} \cdot N$
  - 4:  $s \leftarrow [X_k^2]_{k=1}^N$  ▷ elementwise squared magnitudes
  - 5:  $p \leftarrow \text{argsort}(s)$  ▷ indices that sort  $s$  ascending (smallest first)
  - 6:  $c \leftarrow \text{cumsum}(s[p])$
  - 7:  $t \leftarrow \max\{i : c[i] \leq E_{\text{target}}\}$  ▷ if none,  $t = 0$
  - 8:  $D \leftarrow p[1:t]$  ▷ indices of coefficients we can zero
  - 9:  $\hat{X} \leftarrow X$ ; set  $\hat{X}[D] \leftarrow 0$
  - 10: **return**  $\hat{X}$
- 

**Remarks (role of orthogonality)** If the transform matrix  $A$  for the DCT is orthonormal (i.e.,  $A^\top A = I$ ), then for the pixel block  $x$  and its coefficient vector  $X = Ax$  we have  $\|x\|_2^2 = \|X\|_2^2$  and  $\|x - \hat{x}\|_2^2 = \|X - \hat{X}\|_2^2$ . The second identity is the crucial one used by the algorithm: the *pixel-domain squared error* equals the *sum of squares of discarded DCT coefficients*. This justifies forming  $s_k = X_k^2$  (Step 4), computing cumulative squared energy (Steps 5–6), and comparing that directly to  $E_{\text{target}}$  (Step 3).



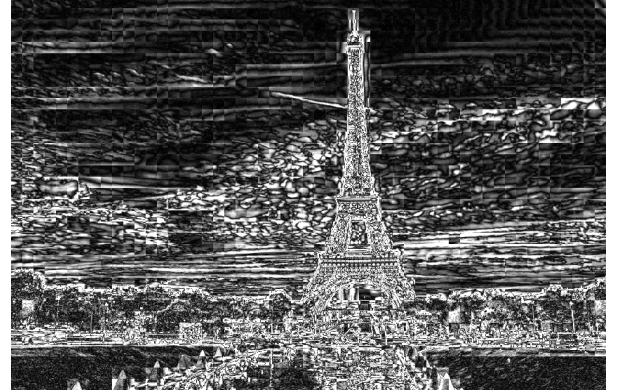
$f1$ 's Block-Wise Lossy Compression ( $\widehat{f1}$ ) in Block comp.



$f1$ 's error image ( $abs(f1 - \widehat{f1})$ ) in Block comp.



$f2$ 's Block-wise Lossy Compression ( $\widehat{f2}$ )



$f2$ 's error image ( $abs(f2 - \widehat{f2})$ )

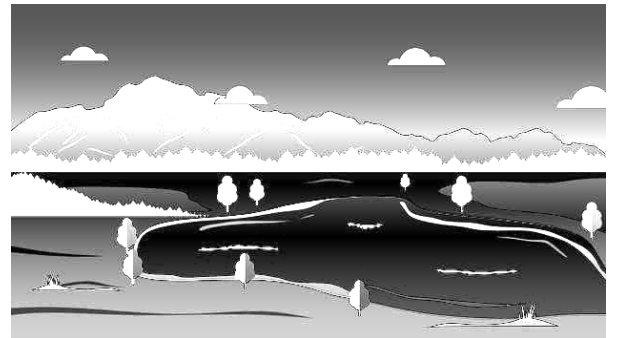
| Target: PSNR $\geq$ 20 dB (RMSE $\leq$ 25.5) [Block-Wise Compression] |          |          |
|-----------------------------------------------------------------------|----------|----------|
|                                                                       | $f1$     | $f2$     |
| PSNR                                                                  | 29.17 dB | 25.49 dB |
| RMSE                                                                  | 8.87     | 13.56    |
| Total coeff.                                                          | 219648   | 474880   |
| Nonzero coeff.                                                        | 872      | 2719     |
| Percentage kept                                                       | 0.40%    | 0.57%    |
| Max error                                                             | 118.43   | 172.12   |

Given the amount of coefficients we have dropped the image quality is still good, Also we can't attain very close to target because we are processing lossy compression blockwise in blocks of size 16 x 16.

### Part-C : Entire Image DCT



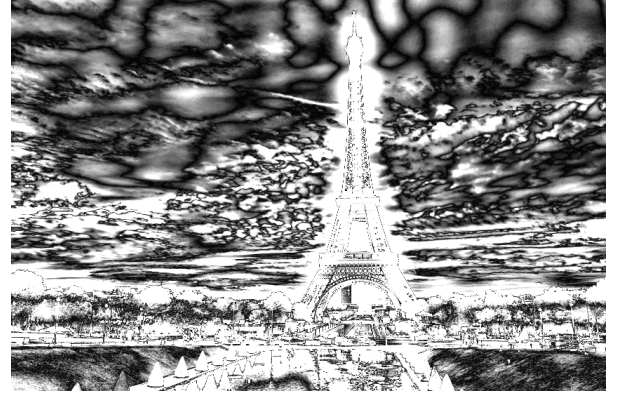
$f1$ 's Global Lossy Compression( $\widehat{f1'}$ )



$f1$ 's error image ( $abs(f1 - \widehat{f1'})$ ) in Global comp.



$f2$ 's Global Lossy Compression( $\widehat{f2'}$ )



$f2$ 's error image ( $abs(f2 - \widehat{f2'})$ ) in Global comp.

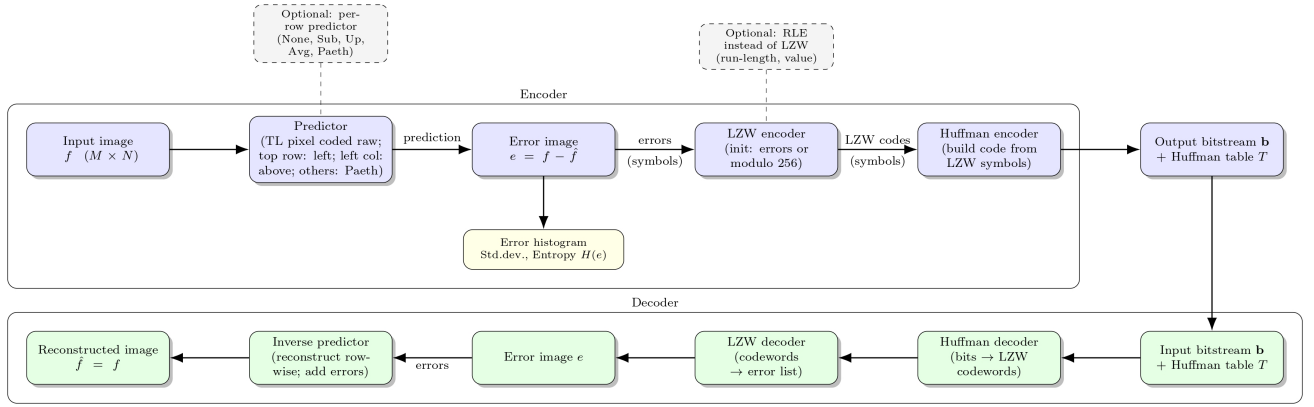
| Target: PSNR $\geq$ 20 dB (RMSE $\leq$ 25.5) [Global Compression] |          |          |
|-------------------------------------------------------------------|----------|----------|
|                                                                   | f1       | f2       |
| PSNR                                                              | 20.92 dB | 20.02 dB |
| RMSE                                                              | 22.93    | 25.43    |
| Total coeff.                                                      | 219648   | 474880   |
| Nonzero coeff.                                                    | 2        | 75       |
| Percentage kept                                                   | 0.00%    | 0.02%    |
| Max error                                                         | 107.12   | 158.57   |

The **Full Image lossy compression** achieves better compression and matches the target PSNR more closely. However, the quality often suffers, for instance, in the cartoon image  $f1$ , almost none of the fine details or textures are preserved. Depending on the specific requirements, either approach can be preferable. If the goal is to retain local features and preserve spatial details while still achieving compression, the **Block-wise strategy** is better suited. On the other hand, if the priority is to strictly meet a target PSNR and maximize compression efficiency, the **Full Image compression** approach is more effective. Below, we highlight some other key differences between these two strategies.

| Block DCT Compression vs Full Image DCT Compression                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Block DCT                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | Full Image DCT                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <b>Pros:</b> <ul style="list-style-type: none"> <li>• Lower computational complexity: blocks can be processed in parallel</li> <li>• Lower memory requirements (only block-size buffers)</li> <li>• Better adaptability to local features</li> <li>• Localized errors (artifacts confined to blocks)</li> <li>• Industry standard (JPEG, MPEG)</li> </ul> <b>Cons:</b> <ul style="list-style-type: none"> <li>• Blocking artifacts at high compression ratios</li> <li>• Less efficient energy compaction than full-image DCT</li> <li>• Discontinuities at block boundaries</li> </ul> | <b>Pros:</b> <ul style="list-style-type: none"> <li>• Better energy compaction: most energy in low-frequency coefficients</li> <li>• No blocking artifacts: smoother reconstructed images</li> <li>• Higher compression ratio for the same PSNR target</li> <li>• Well-suited to smooth, slowly varying content</li> </ul> <b>Cons:</b> <ul style="list-style-type: none"> <li>• High computational complexity (large images)</li> <li>• High memory requirements (need full-image transforms/storage)</li> <li>• Poor adaptability to local features, errors can propagate</li> <li>• Not practical for many real-time or very large-image applications</li> </ul> |
| <b>Conclusion:</b> Block DCT is preferred in practice for its lower complexity, smaller memory footprint and better local adaptability, full-image DCT yields slightly better compression and avoids blocking artifacts but at significant computational and memory cost, making it less practical at scale.                                                                                                                                                                                                                                                                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |

## Question 5: PNG-style Compression Pipeline [*Optional Solved*]

Input Images used here are same as that used in Q4. Here we keep the overall pipeline we need to design for clarity.



Overall Pipeline of Q5

### Part-A : Predictive Encoding using Paeth Predictor

Let  $I \in \{0, \dots, 255\}^{H \times W}$  be the input image,  $E \in \mathbb{Z}^{H \times W}$  the encoded error image, and  $O \in \{0, \dots, 255\}^{H \times W}$  the decoded output. Define constant

$$C := \left\lfloor \frac{255}{2} \right\rfloor = 127.$$

Define the Paeth predictor for scalars  $a, b, c$  by

$$\text{Paeth}(a, b, c) = \arg \min_{t \in \{a, b, c\}} |(a + b - c) - t|.$$

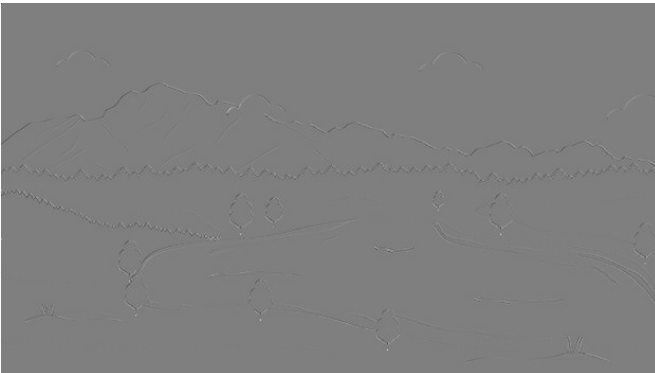
For each pixel index  $(i, j)$  with  $0 \leq i < H$ ,  $0 \leq j < W$  define the predictor

$$\text{pred}_{i,j} = \begin{cases} 0, & i = 0, j = 0, \\ I_{0,j-1}, & i = 0, j > 0, \\ I_{i-1,0}, & i > 0, j = 0, \\ \text{Paeth}(I_{i,j-1}, I_{i-1,j}, I_{i-1,j-1}), & \text{otherwise.} \end{cases}$$

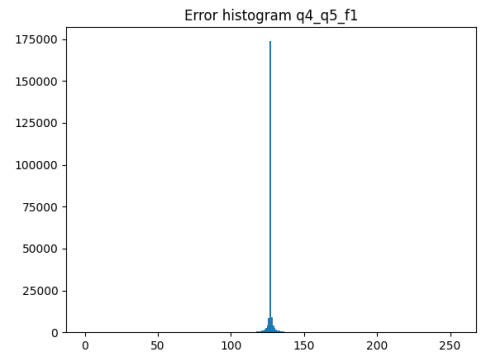
Then the error image entry is

$$E_{i,j} = \text{clip}(I_{i,j} - \text{pred}_{i,j} + C, 0, 255).$$

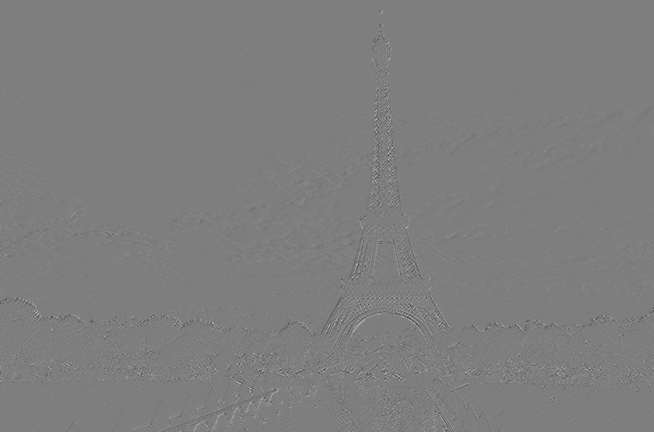
Below we present the results from the predictive error encoding on both  $f_1$  &  $f_2$ . Also note in Part-C on-wards instead of adding  $C(=127)$  and clipping we use modified version where we take a modulo with 255 directly.



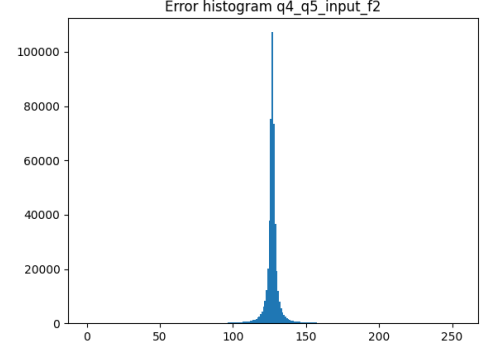
Error image  $e_1$



$e_1$ 's Histogram



Error image  $e_2$



$e_2$ 's Histogram

Empirical data in the table below show that predictive encoding yields higher standard deviation and entropy for natural photographic images than for cartoon images, which contain large constant regions and are more easily predicted by the Paeth predictor.

| Error Image Stats |        |       |
|-------------------|--------|-------|
|                   | $e_1$  | $e_2$ |
| <b>Std. dev.</b>  | 3.0618 | 7.033 |
| <b>Entropy</b>    | 1.605  | 3.888 |

## Part-B : Decoder for Error Images produced from Predictive Encoder

Reconstruct  $O$  in same order used by the encoder. For each  $(i, j)$  define the predictor using already reconstructed neighbours:

$$\widehat{\text{pred}}_{i,j} = \begin{cases} 0, & i = 0, j = 0, \\ O_{0,j-1}, & i = 0, j > 0, \\ O_{i-1,0}, & i > 0, j = 0, \\ \text{Paeth}(O_{i,j-1}, O_{i-1,j}, O_{i-1,j-1}), & \text{otherwise.} \end{cases}$$

Then reconstruct and clamp to the valid range.

$$O_{i,j} = \text{clip}(\widehat{\text{pred}}_{i,j} + (E_{i,j} - C), 0, 255),$$



$f_1$  decoded from  $e_1$



$f_2$  decoded from  $e_2$

We have experimentally verified that there was no error in decoded images.

## Part-C : LZW Encoder & Decoder

### Theorem 1

Let the input to LZW be the constant string  $a^N$  and suppose the initial dictionary contains the single symbol  $a$ . If  $m(N)$  denotes the number of output phrases produced by greedy LZW parsing, then

$$m(N) = \Theta(\sqrt{N}).$$

*Key invariant.* Let the phrase lengths produced by LZW be  $L_1, L_2, \dots, L_m$ . For every  $k \geq 1$ , after producing the first  $k$  phrases the dictionary contains

$$a, a^2, \dots, a^{k+1},$$

and the characters consumed equal

$$\sum_{i=1}^k L_i = \frac{k(k+1)}{2}.$$

*Proof of the invariant (induction).* Base  $k = 1$ :  $L_1 = 1$  and the dictionary becomes  $\{a, a^2\}$ . Inductive step: if the claim holds for  $k$ , then  $a^{k+1}$  is in the dictionary, so the next greedy match is  $a^{k+1}$ , giving  $L_{k+1} = k+1$ ; the dictionary extends to  $a^{k+2}$  and

$$\sum_{i=1}^{k+1} L_i = \frac{k(k+1)}{2} + (k+1) = \frac{(k+1)(k+2)}{2},$$

completing the induction.

Let  $t$  be the largest integer with  $\frac{t(t+1)}{2} \leq N$ . By the invariant, after  $t$  phrases the algorithm has consumed  $\frac{t(t+1)}{2}$  characters; the remainder

$$r := N - \frac{t(t+1)}{2}, \quad 0 \leq r \leq t,$$

is either zero (so  $m(N) = t$ ) or positive (so  $m(N) = t+1$ ). Hence

$$m(N) = t + [r > 0] \in \{t, t+1\}.$$

Solving  $\frac{t(t+1)}{2} \leq N$  gives the positive root

$$t_* = \frac{-1 + \sqrt{1 + 8N}}{2},$$

and  $t = \lfloor t_* \rfloor$ . Since  $t_* = \Theta(\sqrt{N})$ , we obtain  $t = \Theta(\sqrt{N})$  and therefore

$$m(N) = \Theta(\sqrt{N}).$$

□

We experimented with sequence of 100 zeros, the encoding was of length 14. The decoding correctly retrived the input sequence.

## Part-D : Huffman Encoder & Decoder

| Stats                            |                |                |
|----------------------------------|----------------|----------------|
|                                  | f <sub>1</sub> | f <sub>2</sub> |
| Entropy (after prediction + LZW) | 12.207         | 14.499         |
| Average Huffman code length      | 12.253         | 14.522         |
| Total encoded bits               | 277203         | 1833761        |

We can understand these results intuitively as follows. The entropy  $H$  represents the fundamental lower bound on the average number of bits required per symbol, reflecting the inherent uncertainty in the LZW codeword distribution. Huffman coding produces the most efficient possible prefix code, but because it must assign an integer number of bits to each symbol, its average code length  $L_H$  is necessarily slightly larger than  $H$ . This small gap arises from rounding to whole-bit lengths and is completely expected, only non-integer bit-based methods such as arithmetic or ANS coding can reach values arbitrarily close to the entropy.

### Part-E : Full Encoder & Decoder Pipeline

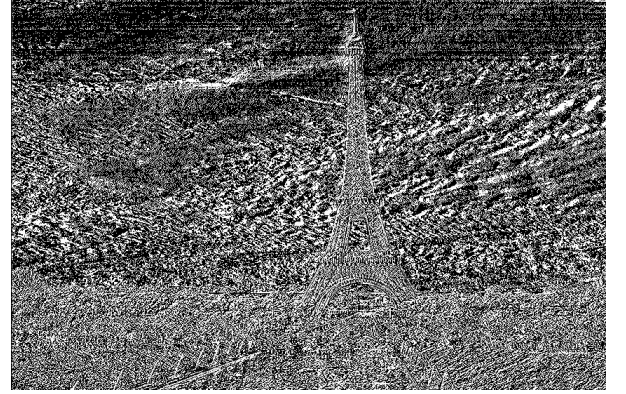
We experimentally verified the decoded image does match the input image exactly. Here we just list down the statistics that we were asked to report.

|                              | Stats |       |
|------------------------------|-------|-------|
|                              | $f_1$ | $f_2$ |
| Prediction compression ratio | 4.984 | 2.057 |
| LZW compression ratio        | 9.709 | 3.761 |
| Actual compression ratio     | 6.339 | 2.072 |

### Part-F : Per-row Optimal Predictive Encoding & RLE



Adaptive Predictor's Error image  $e'_1$



Adaptive Predictor's Error image  $e'_2$

In this section, instead of using only the Paeth predictor, we use an adaptive approach that selects the best predictor for each row from a pool of five options: None, Above, Left, Average (of Above and Left), and Paeth. The predictor that minimizes the entropy of the prediction error for that row is chosen. The selected predictor's label is stored at the beginning of the row in the image, followed by the corresponding error values. As a result, the encoded image has one additional column at the start. The error images generated using this adaptive predictor are shown above.

|                                                            | Stats |        |
|------------------------------------------------------------|-------|--------|
|                                                            | $f_1$ | $f_2$  |
| Adaptive Prediction compression ratio                      | 3.375 | 2.065  |
| LZW compression ratio                                      | 9.643 | 3.792  |
| Actual compression ratio After Huffman Encoding (with LZW) | 6.247 | 2.090  |
| Actual compression ratio After Huffman Encoding (with RLE) | 6.376 | 2.0015 |

If we compare the results from the above table with those in Part E, we observe that using the Adaptive Predictor along with LZW results in only a slightly higher compression ratio for the  $f_2$  case, while the compression ratio actually decreases for the  $f_1$  case. When we use the Adaptive Predictor with RLE, the overall compression ratio for  $f_1$  improves compared to Part E, whereas for  $f_2$  it decreases. This behavior is expected, as RLE performs well on cartoon-like images with large uniform regions but tends to struggle with real photographic images, as seen here.

————— End of Part-II —————